

Full Stack Developer — Technical Test

Multi-Tenant Project Management Application

Experience	1+ Year
Recommended Time	3–4 hours
Backend	Node.js + Express.js
Database	PostgreSQL
Frontend	React
Language	JavaScript or TypeScript
API	REST API

1. Task Overview

Build a small multi-tenant Project Management application using React, Node.js, Express.js and PostgreSQL. The application must implement authentication, RBAC, permission-based authorization, tenant isolation, Project Management CRUD, API rate limiting, and a simple frontend.

The application does not need to be production-scale. Focus on clean implementation, correct authorization, database relationships, validation, API security, and a functional frontend.

2. Technology Requirements

- Backend: Node.js + Express.js
- Database: PostgreSQL
- Frontend: React
- Language: JavaScript or TypeScript
- API: REST API
- ORM/query library: candidate's choice (Prisma, Sequelize, TypeORM, Knex, or raw SQL)
- Frontend styling: candidate's choice

3. User Roles

The system must support the hierarchy:

SUPER ADMIN → ADMIN → AGENT

Super Admin

- System-level user and not restricted to one tenant.
- Can view and manage users across all tenants.
- Can create and manage Admin users.
- Can view, create, update and delete Projects across all tenants.
- Can manage the permissions available to Admins.

Admin

- Belongs to one tenant.
- Can create, update, enable and disable Agents belonging to the same tenant.
- Can view, create, update and delete Projects within the same tenant.
- Cannot create or modify Super Admins.
- Cannot create another Admin.
- Cannot access another tenant's data.

Agent

- Belongs to one tenant.
- Can view Projects belonging to the same tenant.

- Cannot create, update or delete Projects unless the relevant permission is explicitly granted.
- Cannot manage users, roles or permissions.
- Cannot access data belonging to another tenant.

4. Permission System

Implement permissions separately from roles. Authorization should be based on permissions where appropriate, rather than hardcoding every API around role names.

```
users.read
users.create
users.update
users.disable
projects.read
projects.create
projects.update
projects.delete
permissions.manage
```

Example: an Agent may have projects.read and projects.update but not projects.delete. An Admin may have project-management permissions but must not be able to create another Admin.

5. Multi-Tenant Data Isolation

```
Tenant A
Admin A
Agent A1
Project A1
Project A2

Tenant B
Admin B
Agent B1
Project B1
```

- Admin A can access only Tenant A data.
- Agent A1 can access only Tenant A data.
- Admin A must not access or modify Project B1.
- Agent A1 must not access or modify Project B1.
- Changing an ID in the URL must not bypass tenant authorization.
- Tenant isolation must be enforced by the backend.
- Do not blindly trust tenantId supplied by the frontend; derive tenant scope from the authenticated user.

6. Authentication

```
POST /auth/login
```

- Use JWT or another reasonable authentication mechanism.
- Passwords must be securely hashed and never stored in plain text.
- Protected APIs must require authentication.
- Unauthenticated requests must return 401 Unauthorized.

7. Project Management — Backend

The core business entity is a Project. A Project represents a customer/business project managed within a tenant.

7.1 Project Fields

Field	Type	Required	Description
id	UUID	Yes	Unique identifier for the project.
name	String	Yes	Name of the project.
address	String	Yes	Address/location associated with the project.
useCase	String	Yes	Brief description of what the project is intended for.
status	Enum/String	Yes	Current project status.
tenantId	UUID	Yes	Tenant/company that owns the project.

Field	Type	Required	Description
createdAt	Timestamp	Yes	Project creation time.
updatedAt	Timestamp	Yes	Last modification time.

7.2 Project Status

```
ACTIVE
INACTIVE
DRAFT
```

The candidate may represent these as a PostgreSQL enum or as a validated string.

7.3 Example Project

```
{
  "name": "WaziApp",
  "address": "Infopark, Thrissur, Kerala",
  "useCase": "WhatsApp-based customer communication and support",
  "status": "ACTIVE"
}
```

id, tenantId, createdAt and updatedAt should be managed by the backend/database and should not be blindly trusted from the client.

7.4 Required APIs

Method	Endpoint	Purpose	Permission
POST	/projects	Create project	projects.create
GET	/projects	List accessible projects	projects.read
GET	/projects/:id	Get one project	projects.read
PUT	/projects/:id	Update project	projects.update
DELETE	/projects/:id	Delete project	projects.delete

7.5 Create Project

```
POST /projects
{
  "name": "WaziApp",
  "address": "Infopark, Thrissur, Kerala",
  "useCase": "WhatsApp customer communication",
  "status": "ACTIVE"
}
```

The backend must determine tenantId from the authenticated user's tenant context.

7.6 List and Get Projects

GET /projects must return only Projects the authenticated user is authorized to access. Admin and Agent users are restricted to their own tenant. Super Admin may access Projects across tenants.

GET /projects/:id must verify permission and tenant/resource access. Changing the ID must never bypass tenant authorization.

7.7 Update Project

```
PUT /projects/:id
{
  "name": "WaziApp Customer Platform",
  "address": "Infopark, Thrissur, Kerala",
  "useCase": "WhatsApp customer communication and support",
  "status": "ACTIVE"
}
```

Requires projects.update and appropriate tenant/resource authorization.

7.8 Delete Project

```
DELETE /projects/:id
```

Requires projects.delete and appropriate tenant/resource authorization. Successful deletion may return 204 No Content.

7.9 Project Validation

- name is required and non-empty.
- address is required and non-empty.
- useCase is required and non-empty.

- status must be ACTIVE, INACTIVE or DRAFT.
- Malformed project IDs must be rejected.
- Invalid input should return 400 Bad Request.

8. Project Permission Behaviour

Project CRUD is permission-driven rather than based only on the user's role.

User	Read	Create	Update	Delete
Super Admin	✓	✓	✓	✓
Admin	✓	✓	✓	✓
Agent — default	✓	✗	✗	✗
Agent — permissions granted	✓	As granted	As granted	As granted

Example: Agent A1 has projects.read and projects.update, but not projects.delete. The Agent can view and edit projects in Tenant A but cannot delete them.

If an Agent directly calls a protected API without the required permission, the backend must return 403 Forbidden. Frontend button visibility alone is not a security control.

9. Frontend — React

Build a simple React frontend that consumes the REST APIs. The UI does not need to be highly styled; functionality and permission-aware behaviour are the priority.

Login Page

- Email and password fields.
- Login action.
- After successful authentication, show the application dashboard.

Projects Page

Display the user's accessible Projects and provide actions according to permissions.

```

Projects
-----
Name Address Use Case Status Actions
-----
WaziApp Infopark WhatsApp Active Edit
CRM System Thrissur CRM Active Edit

[Create Project]
```

- Users without projects.create should not see Create Project.
- Users without projects.update should not see Edit.
- Users without projects.delete should not see Delete.
- Frontend checks are for UI behaviour only; backend remains the source of truth.

User Management

- Provide a simple user management screen for users with relevant permissions.
- At minimum, display users and their roles.
- Admins should be able to create, update, enable and disable Agents in their own tenant.

Permission Management

- Provide a simple permission management screen for Super Admin.
- Allow selecting permissions for the Admin role and saving changes.
- Permission changes should affect backend authorization, not just frontend UI.

10. API Rate Limiting

```
POST /auth/login → maximum 5 requests per IP address within 15 minutes
```

When exceeded, return HTTP 429 Too Many Requests.

```
General API → approximately 100 requests per minute per IP/client
```

Implement the rate limiting using an Express-compatible approach.